

Partie1 TP N°3

Application web (IHM)

Durée max conseillée : 2h

Node-RED

Partie lecture durée conseillée : 15 min

Objectif général

Ce TP prolonge les TP1 et TP2. Il aborde la construction d'une interface homme-machine (IHM) web avec Node-RED permettant de visualiser et de piloter à distance les échanges de données avec la carte Arduino via liaison série.

1 Préambule

1.1 Objectifs pédagogiques

- Installer et prendre en main Node-RED pour construire une interface web sans développement web classique.
- Établir une communication série bidirectionnelle entre une carte Arduino et Node-RED.
- Construire un tableau de bord affichant des mesures de capteurs et pilotant des actionneurs (LED tout-ou-rien et LED PWM).

1.2 Prérequis

Les connaissances développées dans les TP1 et TP2 sont considérées comme acquises.

- Utiliser l'IDE Arduino : créer, compiler et téléverser un programme (TP1).
- Configurer une entrée/sortie numérique et utiliser pinMode(), digitalRead(), digitalWrite() (TP1).
- Utiliser une interruption externe : attachInterrupt(), digitalPinToInterrupt(), une ISR, une variable volatile (TP2).
- Utiliser le Moniteur Série (TP1) et structurer une trame JSON (TP2).
- Comprendre les principes élémentaires du C/C++ : variables, fonctions, conditions, boucles.

1.3 Introduction

1.3.1 Problématique

2 Matériel nécessaire

2.1 Carte de développement et poste de travail

La plateforme utilisée est la même que dans les TP1 et TP2 : Arduino Uno (ATmega328P). Le poste de travail doit en plus disposer de Node.js et de Node-RED (installation détaillée dans le TP).

Élément	Rôle
Arduino Uno	Microcontrôleur et plateforme de développement
Câble USB	Alimentation, programmation et liaison série avec le PC
Ordinateur (Windows)	IDE Arduino, Node.js, npm, Node-RED
DHT11 Grove	Mesure de température et d'humidité (réutilisé du TP2)
Potentiomètre Grove	Entrée analogique pour le tableau de bord
Bouton poussoir Grove	Entrée numérique transmise dans la trame JSON
2 LEDs (Grove ou simples)	LED 7 : tout-ou-rien — LED 6 : commande PWM

2.2 Broches utilisées

Fonction	Broche proposée	Remarque
LED de test Timer2	LED_BUILTIN (D13)	Utilisée pour la partie 1 (base de temps)
LED 7	D7	Commande tout-ou-rien depuis Node-RED
LED 6	D6	Commande PWM (0-255) depuis Node-RED
Bouton poussoir	D2	Lecture d'état, transmise dans la trame JSON
DHT11 / Potentiomètre	D4	Communication DHT11 microcontrôleur

3 Prise en main de Node-RED, créer une application WEB (IHM)

3.1 Introduction.

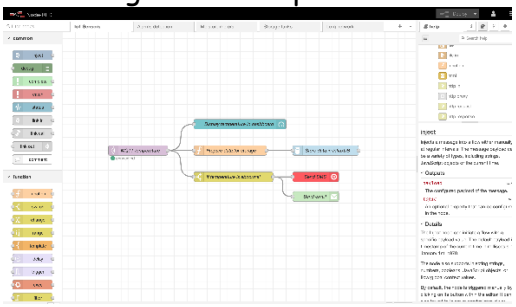
Dans de nombreux projets, l'IHM est constituée d'une application web accessible depuis un navigateur. Il n'est pas toujours aisé de programmer le lien entre le matériel (capteurs et/ou actionneurs) et la page web fournie à l'utilisateur.
Une solution consiste à utiliser un script Python dont l'exécution peut être planifiée avec Cron, pour interagir avec le matériel, lire les données des capteurs et les stocker

dans une base de données comme MySQL. Un serveur web comme Apache2, via une page web php, met à disposition des utilisateurs les informations. Cette solution nécessite la mobilisation de nombreuses technologies et plusieurs langages de programmation, contraignant les développeurs du projet à retarder la mise œuvre d'un prototype pour se former.

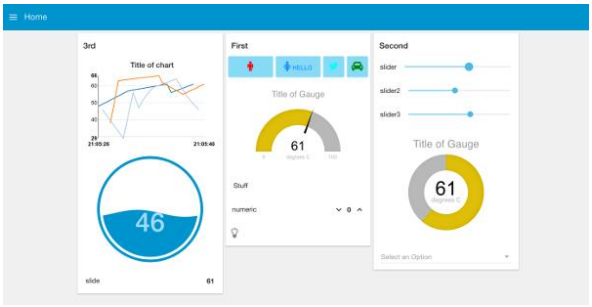
Combiné avec une solution matérielle constituée d'un PC et d'une carte Arduino, Node Red se révèle être une alternative très intéressante :

Node-RED est un outil puissant pour construire des applications de l'Internet des Objets (IoT) en mettant l'accent sur la simplification de la programmation qui se fait grâce à des blocs de code prédéfinis, appelés «nodes» pour effectuer des tâches. Il utilise une approche de programmation visuelle qui permet aux développeurs de connecter les blocs de code ensemble. Les nœuds connectés, généralement une combinaison de nœuds d'entrée, de nœuds de traitement et de nœuds de sortie, lorsqu'ils sont câblés ensemble, constituent un «flow».

Programmation par nœuds



IHM



4 Travail pratique n°1 — Installation de Node-RED sous Windows

Objectif : installer Node-RED sur un ordinateur Windows et vérifier son bon fonctionnement.

4.1 Vérifier si Node.js est installé sur le poste de travail

1- Ouvrir l'invite de commandes Windows (cmd).
2- Saisir la commande suivante : `node -v`
3- Vérifier la version de Node.js installée.

La version disponible est indiquée sur le site officiel de Node.js :
https://nodejs.org/en/download/
Privilégier une **version LTS (Long Term Support)** de Node.js.

4- Deux cas sont possibles :

- Si Node.js est installé et que la version correspond à la dernière version LTS disponible, passer à l'étape suivante.
- Si Node.js n'est pas installé ou si une version plus ancienne est installée, télécharger et installer la dernière version LTS en utilisant **Windows Installer (.msi)**.

4.2 Installer ou mettre à jour npm

npm est installé automatiquement avec Node.js.

a. Ouvrir l'invite de commandes Windows (cmd).

b. Vérifier la version de npm installée : `npm -v`

c. Mettre à jour npm vers la dernière version disponible : `npm install -g npm@latest`

d. Vérifier la nouvelle version de npm : `npm -v`

```
npm -v                    # vérifier la version installée
npm install -g npm@latest # mettre à jour npm
npm -v                    # vérifier la nouvelle version
```

4.2.1 Installer ou mettre à jour Node-RED

a. Ouvrir l'invite de commandes Windows (cmd).

b. Installer ou mettre à jour Node-RED avec la commande :

npm install -g node-red

Cette commande permet :

- d'installer Node-RED s'il n'est pas encore installé ;
- de mettre à jour Node-RED s'il est déjà installé.

c. Vérifier la version de Node-RED :

node-red -version

```
npm install -g node-red    # installe ou met à jour Node-RED
node-red --version
```

4.2.2 Lancer Node-RED

Dans l'invite de commandes, saisir : `node-red`

Attendre l'apparition du message indiquant que le serveur est démarré.

```
C:\Users\UserName>node-red
3 Aug 18:11:19 - [info]

Welcome to Node-RED
===================

3 Aug 18:11:19 - [info] Node-RED version: v5.0.4
3 Aug 18:11:19 - [info] Node.js  version: v24.16.0
3 Aug 18:11:19 - [info] Windows_NT 10.0.19045 x64 LE
3 Aug 18:11:19 - [info] Loading palette nodes
3 Aug 18:11:25 - [info] Dashboard version 3.6.6 started at /ui
3 Aug 18:11:26 - [info] Settings file  : \Users\ezoua\.node-
red\settings.js
3 Aug 18:11:26 - [info] Context store  : 'default' [module=memory]
3 Aug 18:11:26 - [info] User directory : \Users\ezoua\.node-red
3 Aug 18:11:26 - [warn] Projects disabled :
editorTheme.projects.enabled=false
3 Aug 18:11:26 - [warn] Flows file name not set. Generating name
using hostname.
3 Aug 18:11:26 - [info] Flows file     : \Users\ezoua\.node-
red\flows_DESKTOP-DM5IGV2.json
3 Aug 18:11:26 - [info] Server now running at
http://127.0.0.1:1880/
3 Aug 18:11:26 - [warn]
```

```
------------------------------------------------------------------
------
Your flow credentials file is encrypted using a system-
generated key.

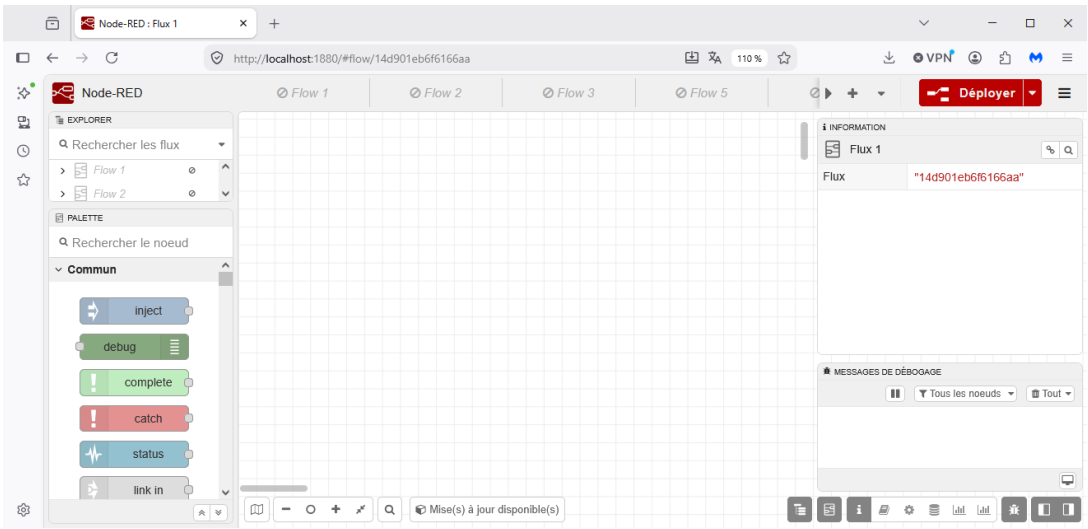
If the system-generated key is lost for any reason, your
credentials
file will not be recoverable, you will have to delete it and re-
enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
------------------------------------------------------------------
--

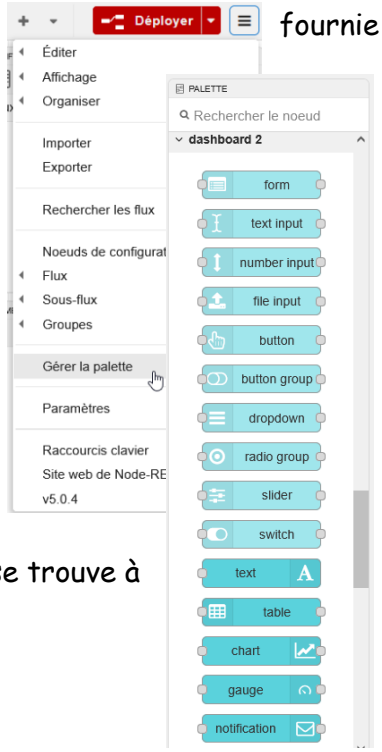
3 Aug 18:11:26 - [info] Starting flows
3 Aug 18:11:26 - [info] Started flows
3 Aug 18:11:26 - [info] [mqtt-broker:3054a0b2.3910c] Connected to
broker: mqtt://192.168.1.107:1883
```

Ouvrir ensuite un navigateur Internet et saisir : http://localhost:1880 ou http://ipAdress:1880 exemple http://192.168.1.113:1880 (ce qui permet d'avoir accès à son poste de travail à distance)

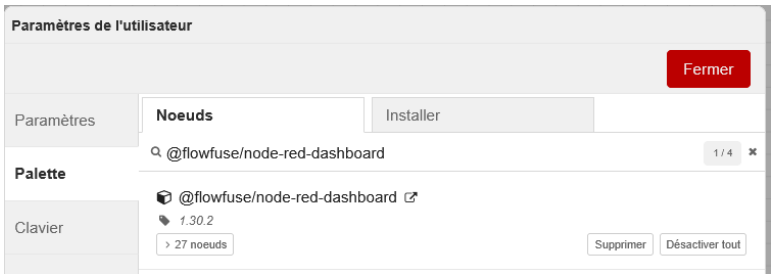
L'interface de Node-RED doit alors apparaître.



La palette de Nodes d'origine est déjà bien fournie mais il en manque au moins une essentielle qui va permettre de réaliser l'interface web.



Ouvrir le gestionnaire de palettes :
Menu en haut à droite→gérer palette
Installer la palette : `@flowfuse/node-red-dashboard`



La palette dashboard2 apparaît dans le menu palette qui se trouve à gauche.

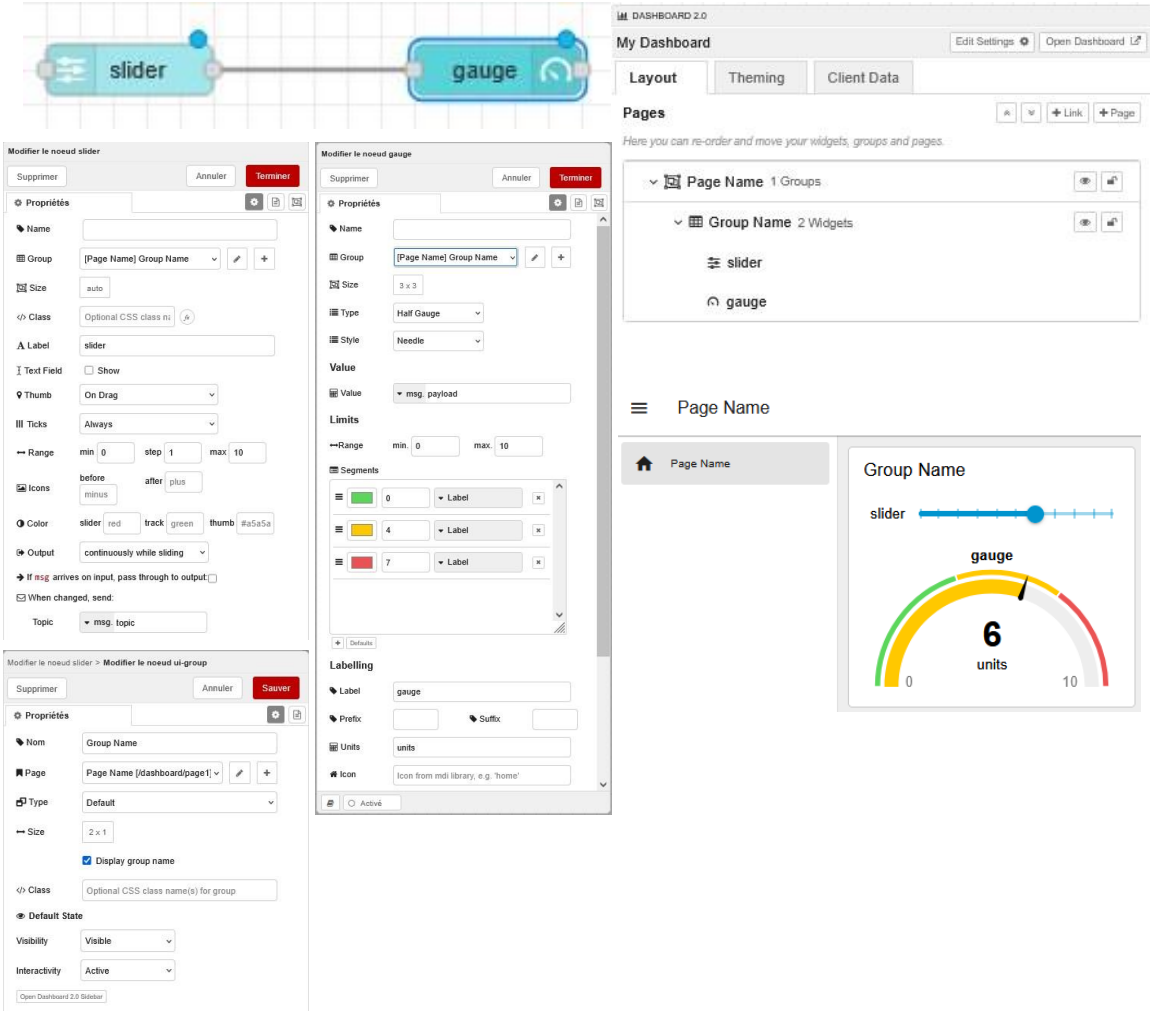
5 Travail pratique n°2 — Premier flow

Objectif : réaliser un premier flow simple (slider + gauge), le personnaliser, puis l'enrichir d'un historique et d'une information textuelle horodatée.

Commençons par réaliser un flow très simple composé d'un node Slider et d'un node Gauge. Pour modifier la configuration par défaut des nodes, il suffit de double-cliquer dessus.
Pour que ces composants soient bien disposés sur la page web, il faut les mettre dans un Tab, puis dans un Groupe, dans la boite de sélection du groupe, cliquez sur Add new ui_group, puis créer un groupe (ici : "Le groupe") et faite de même pour le Tab (ici : "Test").
Cliquez ensuite sur le bouton Done, puis déployer le Flow. On accède au site web généré par l'url : http://ipAdress:1880/dashboard exemple: http://192.168.1.107:1880/dashboard

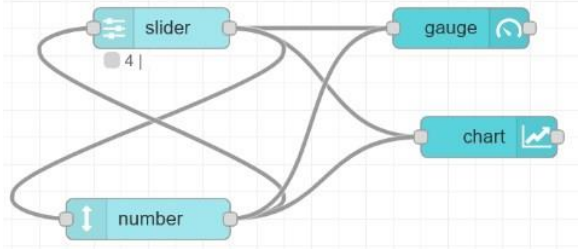
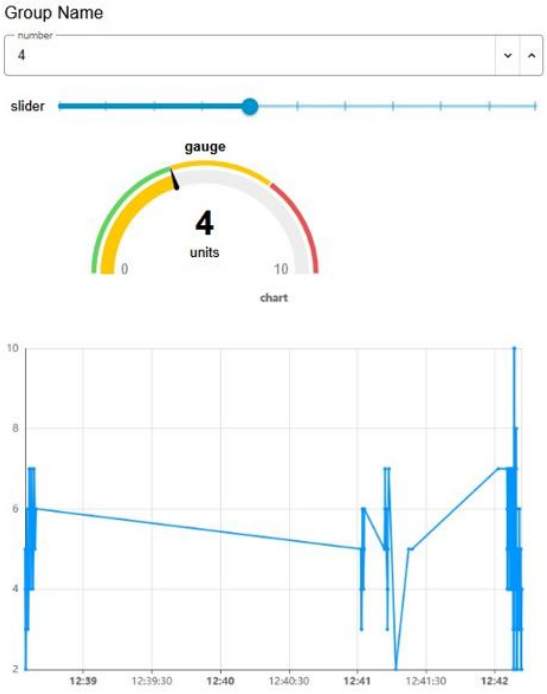
Travail demandé

1. Saisir un premier flow composé d'un node slider et d'un node gauge, en laissant les paramètres par défaut.
2. Personnaliser les noms de page, de groupe, du slider et de la gauge.
3. Ajouter un node chart pour conserver un historique, et un node number pour saisir un chiffre.
4. Ajouter une information textuelle en bas du groupe (valeur du slider, date et heure de la dernière modification). Un simple node Text ne suffit pas : il faut construire le message avec un node Function.

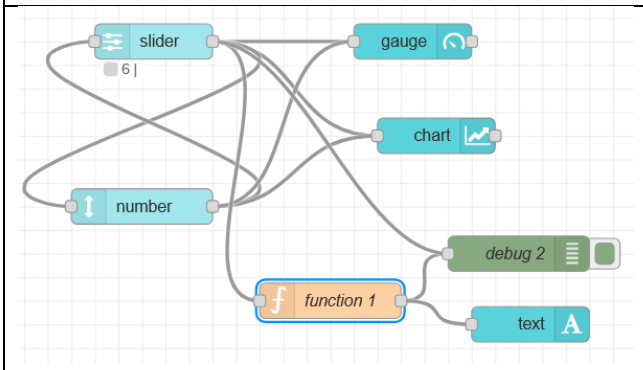
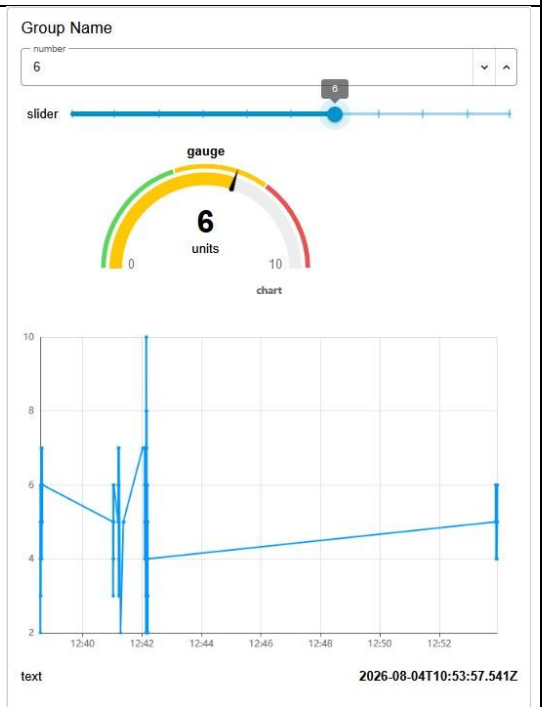


A vous de rechercher pour modifier les éléments nom de page, nom du groupe, nom du slider, nom de la gauge pour obtenir un premier flow personnalisé.

Améliorer votre premier flow en ajoutant un node (nœud) chart pour conserver un historique ainsi qu'une possibilité de choisir un chiffre.

Flow	Interface WEB
	

Rajouter une information textuelle en bas du groupe. Cette information contiendra par exemple la valeur du **slider**, la date et l'heure de modification. L'ajout d'un node **Text** ne suffit pas car il permet d'ajouter un label et le contenu d'un message

Flow	Interface WEB
	

5.1 Le node Function

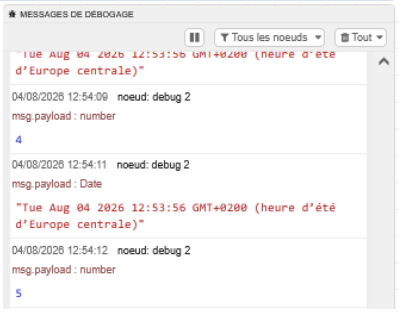
Le node Function permet d'exécuter du code JavaScript sur le message entrant pour produire un ou plusieurs messages sortants. Les messages transmis entre nodes sont des objets ; l'attribut payload ("charge utile") contient l'information transmise. Un node Debug permet d'observer ce payload.

```
var now    = new Date();
var year   = now.getFullYear();
var month  = now.getMonth() + 1;
var day    = now.getDate();
var hour   = now.getHours();
var minute = now.getMinutes();
var second = now.getSeconds();

msg.payload = day + "/" + month + "/" + year + " à " +
              hour + ":" + second + " => " + msg.payload;

return msg;
```

A noter : Les messages transmis dans les flux d'entrée et de sortie des nodes sont des objets. C'est l'attribut **payload**, soit "**charge utile**" en français qui contient l'information à transmettre. Pour observer cette charge utile de message vous pouvez utiliser un node **Debug**.



Pour aller plus loin, je vous conseille la lecture de user guide et faire quelques tutoriels de tutorial de Node RED.
https://nodered.org/docs/

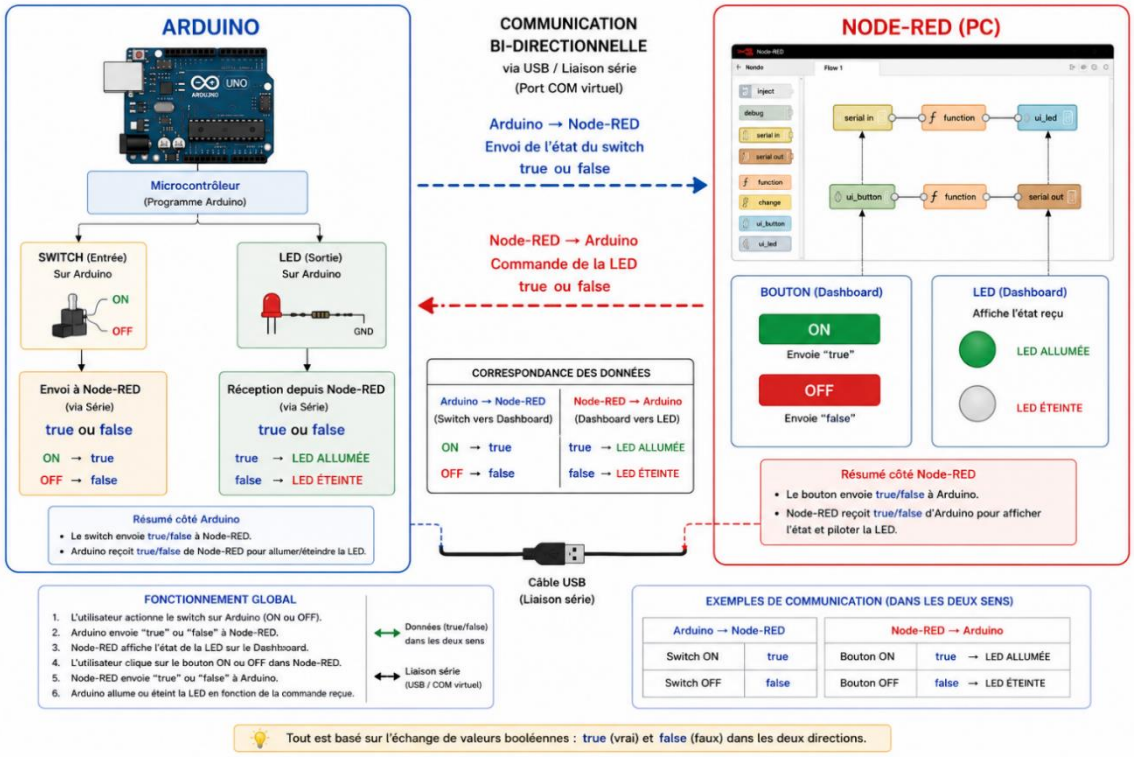
5.2 Questionnaire

1. Quel est le rôle de l'attribut payload dans un message Node-RED ?
2. À quoi sert un node Debug ?
3. Pourquoi utiliser un node Function plutôt que de modifier directement le node Gauge ?
4. Quelle est la différence entre un Tab et un Groupe dans le Dashboard ?

6 Travail pratique n°3 — Communication série avec l'Arduino

Objectif : installer dans Node-RED le module permettant de communiquer avec une carte Arduino par l'intermédiaire de son port USB série, et échanger un état booléen dans les deux sens.

La communication sera réalisée selon le principe suivant :

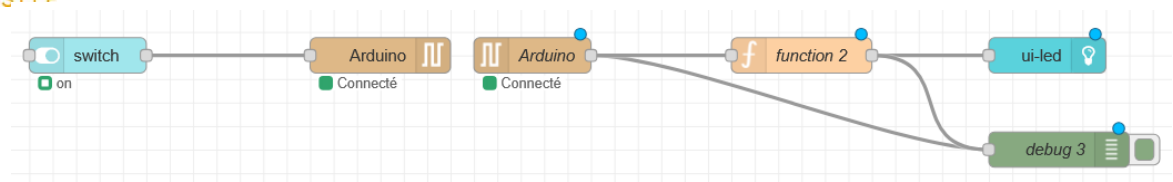


Le port USB de l'Arduino est utilisé comme **liaison série** entre la carte et l'ordinateur.

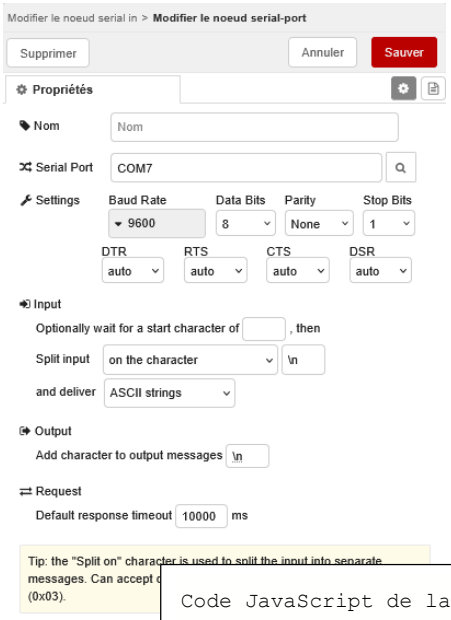
Le port USB de l'Arduino est utilisé comme liaison série entre la carte et l'ordinateur. Un switch côté Node-RED envoie true/false à l'Arduino, qui pilote une LED en conséquence ; en retour, l'Arduino envoie l'état d'un interrupteur physique, affiché par une LED virtuelle côté dashboard.

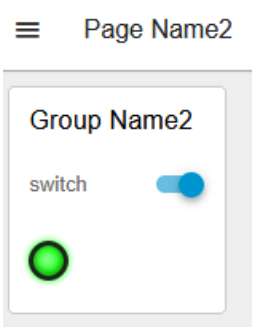
6.1.1 Travail demandé

1- Programmer l'Arduino avec le programme `echo_serial.ino` qui permet de tester le bon fonctionnement de votre flow.
2- Choisir les connexions des composants. Câbler l'interrupteur et la Led.
3- Installer le module de communication série :
   a. Dans Node-RED, ouvrir le menu :
   b. ☰ → **Manage palette**
   c. Puis sélectionner l'onglet :
   d. **Installer.**
   e. Dans le champ de recherche, saisir :
      i. `node-red-node-serialport`
   f. Cliquer sur installer.
4- Installer la palette : `@flowfuse/node-red-dashboard-2-ui-led`
5- Saisir le flow et saisir les configurations suivantes dans une deuxième page. Attention il faut adapter votre propre numéro de port COM.









Code JavaScript de la fonction2 :

```
msg.payload = msg.payload.toString().trim();

if (msg.payload === "true") {
    msg.payload = true;
} else if (msg.payload === "false") {
    msg.payload = false;
}

return msg;
```

6- Tester le bon fonctionnement de votre flow.
7- Décrire le fonctionnement.
8- Ecrire le programme d'interaction côté Arduino.
9- Tester le bon fonctionnement de votre programme.

6.1.2 Conversion du payload reçu

La liaison série ne transporte que du texte : la chaîne reçue doit être explicitement convertie en booléen avant d'être exploitée par le node ui-led.

```
msg.payload = msg.payload.toString().trim();

if (msg.payload === "true") {
    msg.payload = true;
} else if (msg.payload === "false") {
    msg.payload = false;
}

return msg;
```

6.2 Questionnaire

1. Pourquoi faut-il convertir la chaîne reçue ("true"/"false") en booléen dans le node Function ?

2. Que transporte concrètement le port série entre l'Arduino et Node-RED ?

3. Quel rôle jouent respectivement les nodes serial in et serial out ?

7 Projet de synthèse — Communication bidirectionnelle complète

7.1 Contexte

Dans ce travail, nous allons mettre en œuvre une communication bidirectionnelle entre une carte Arduino et un ordinateur via une liaison série USB.
L'Arduino acquiert les informations de plusieurs capteurs et entrées : température et humidité (DHT11), position du potentiomètre et état du bouton. Ces données sont regroupées dans une trame JSON transmise périodiquement à l'ordinateur. Sur le PC, Node-RED reçoit, décode et visualise ces informations dans un tableau de bord, et peut envoyer des commandes JSON à l'Arduino pour piloter LED 7 (tout-ou-rien) et LED 6 (PWM).

Objectif : mettre en évidence une chaîne complète :

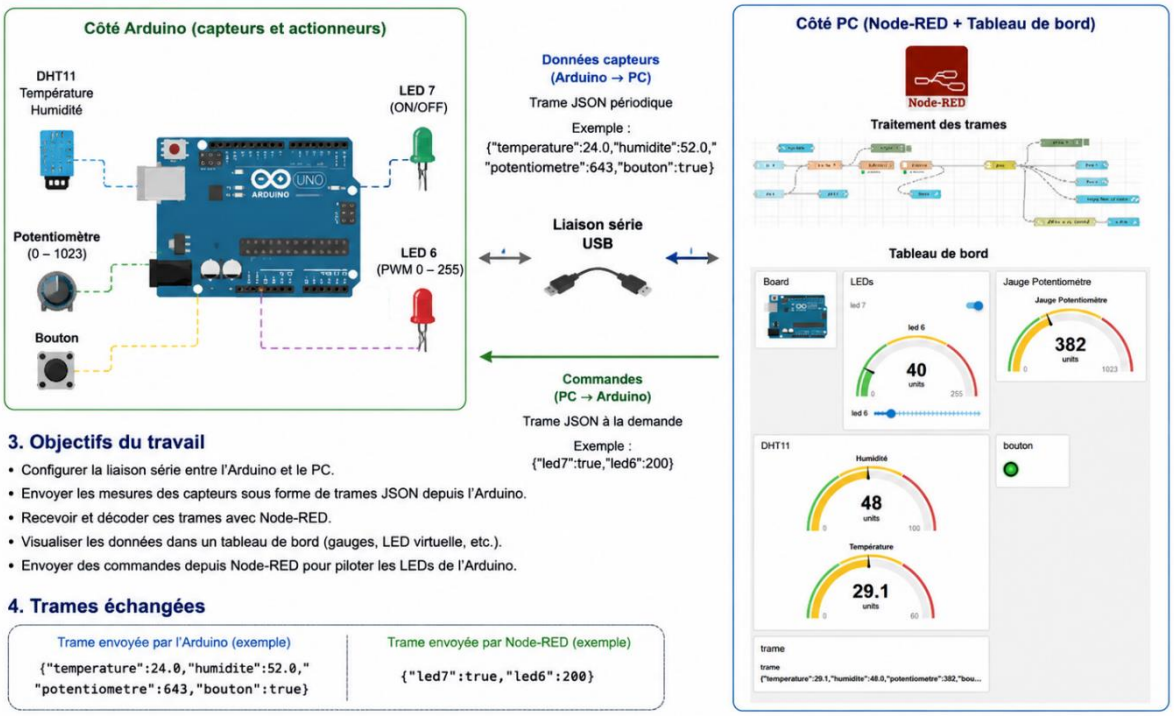
acquisition → communication → traitement → visualisation → commande.

Sur le PC, Node-RED reçoit, décode et exploite ces trames afin de visualiser les informations dans un tableau de bord. Il peut également envoyer des commandes JSON à l'Arduino pour piloter les deux LEDs : LED 7 en tout-ou-rien et LED 6 en PWM.

1. Introduction

Dans ce travail, nous mettons en place une communication bidirectionnelle entre une carte Arduino et un ordinateur via la liaison série (USB). L'Arduino, équipé de capteurs (DHT11 pour la température et l'humidité, potentiomètre, bouton) et de deux LEDs (une tout ou rien et une PWM), envoie périodiquement des mesures au PC sous forme de trames JSON. Le PC, via Node-RED, visualise ces données dans un tableau de bord et peut également envoyer des commandes à l'Arduino pour piloter les LEDs.

2. Contexte du système



3. Objectifs du travail

- Configurer la liaison série entre l'Arduino et le PC.
- Envoyer les mesures des capteurs sous forme de trames JSON depuis l'Arduino.
- Recevoir et décoder ces trames avec Node-RED.
- Visualiser les données dans un tableau de bord (gauges, LED virtuelle, etc.).
- Envoyer des commandes depuis Node-RED pour piloter les LEDs de l'Arduino.

4. Trames échangées

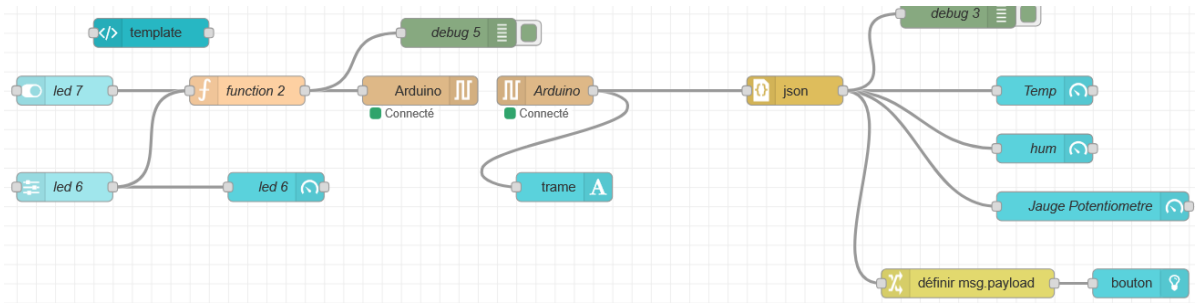
Trame envoyée par l'Arduino (exemple)	Trame envoyée par Node-RED (exemple)
`{"temperature":24.0,"humidite":52.0," "potentiometre":643,"bouton":true}`	`{"led7":true,"led6":200}`

Objectifs :

1- Établir la liaison série Arduino ↔ PC.
2- Recevoir et décoder une trame JSON dans Node-RED.
3- Extraire les différentes données de la trame.
4- Afficher les données dans le tableau de bord.
5- Générer une trame de commande depuis Node-RED.
6- Décoder cette trame dans l'Arduino.
7- Commander les LEDs en tout-ou-rien et en PWM.

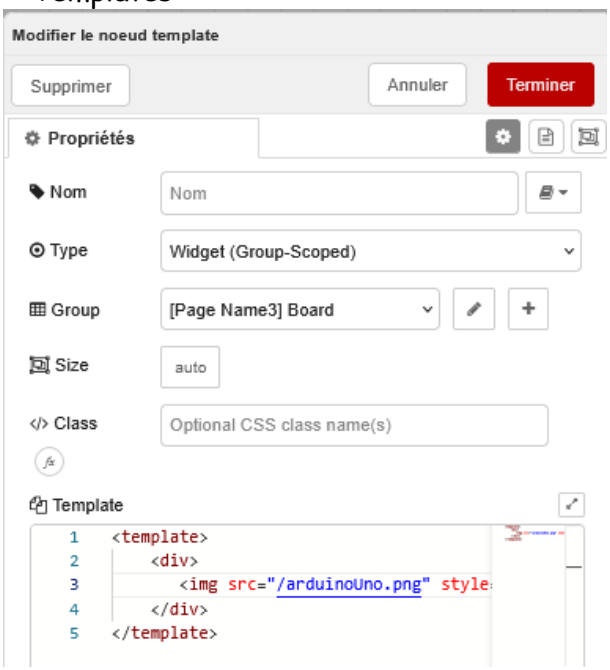
Travail demandé

1- Programmer l'Arduino avec le programme `comBidirectionnelle.ino`
2- Choisir les connexions des composants. Câbler.
3- Saisir Flow avec toutes les configurations indiquées



Les configurations :

- Templates

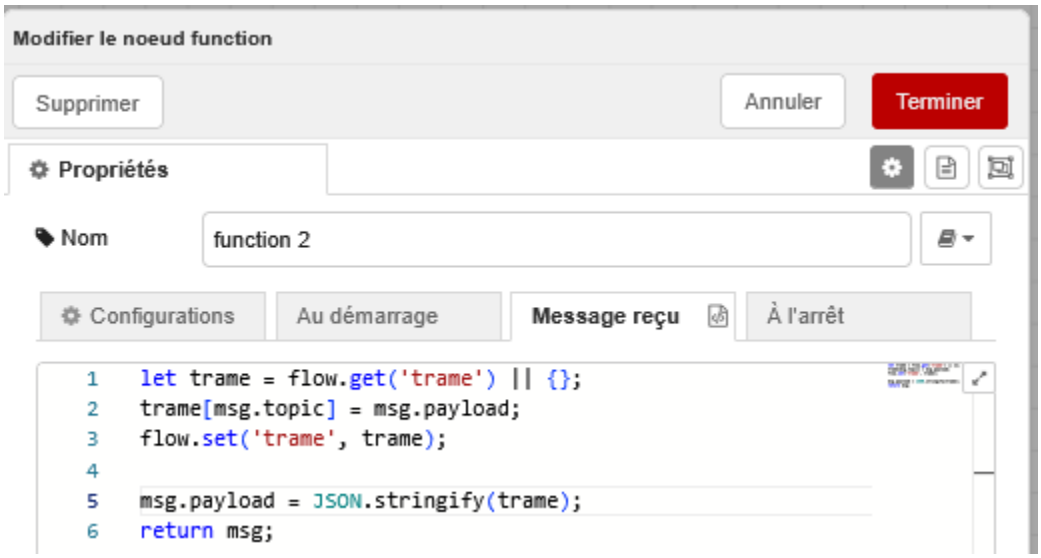


Pour que le template soit visible dans le dashboard, il faut configurer Node-RED. Avec un éditeur de texte, ouvrir le fichier `settings.js` du répertoire `C:\Users\UserName\.node-red` de votre PC. Ajouter la ligne `httpStatic: 'C:\\Users\\ UserName \\.node-red\\node-red-static\\',` . Sauvegarder les modifications. Créer le répertoire de référence `C:\Users\ UserName \.node-red\node-red-static`. Copier, L'image du templates, fournie en annexe ou dans le dossier img, dans ce répertoire.

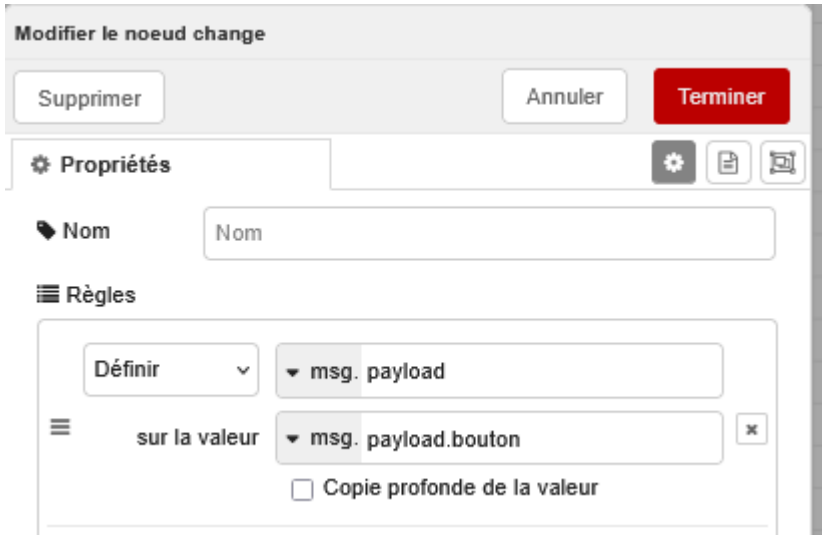
- Configuration Arduino envoie et réception des données :



- Function 2

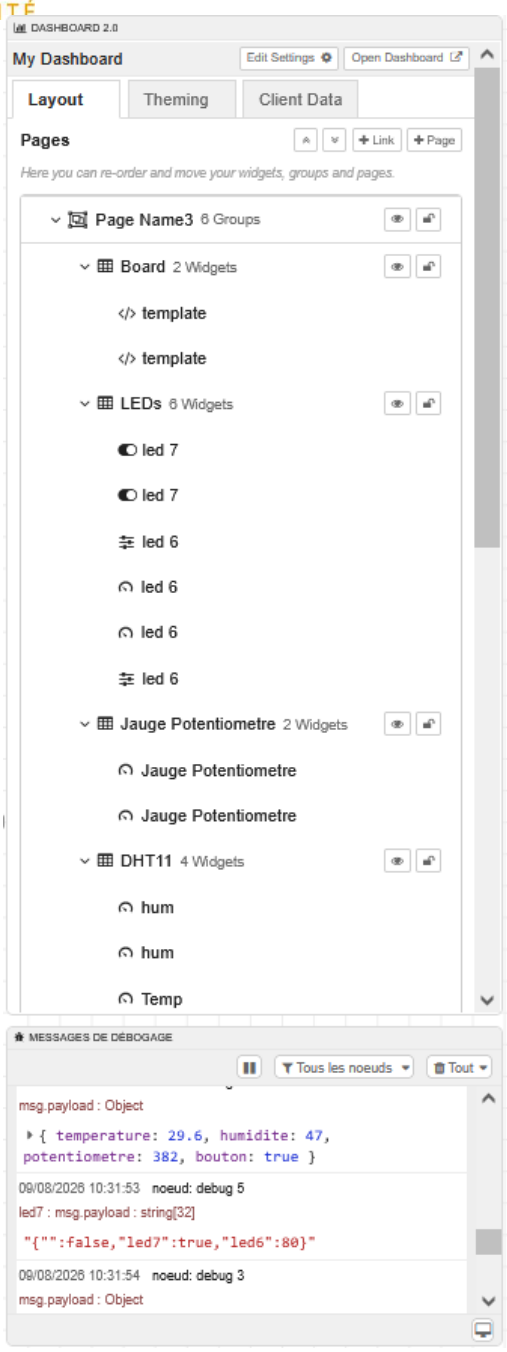


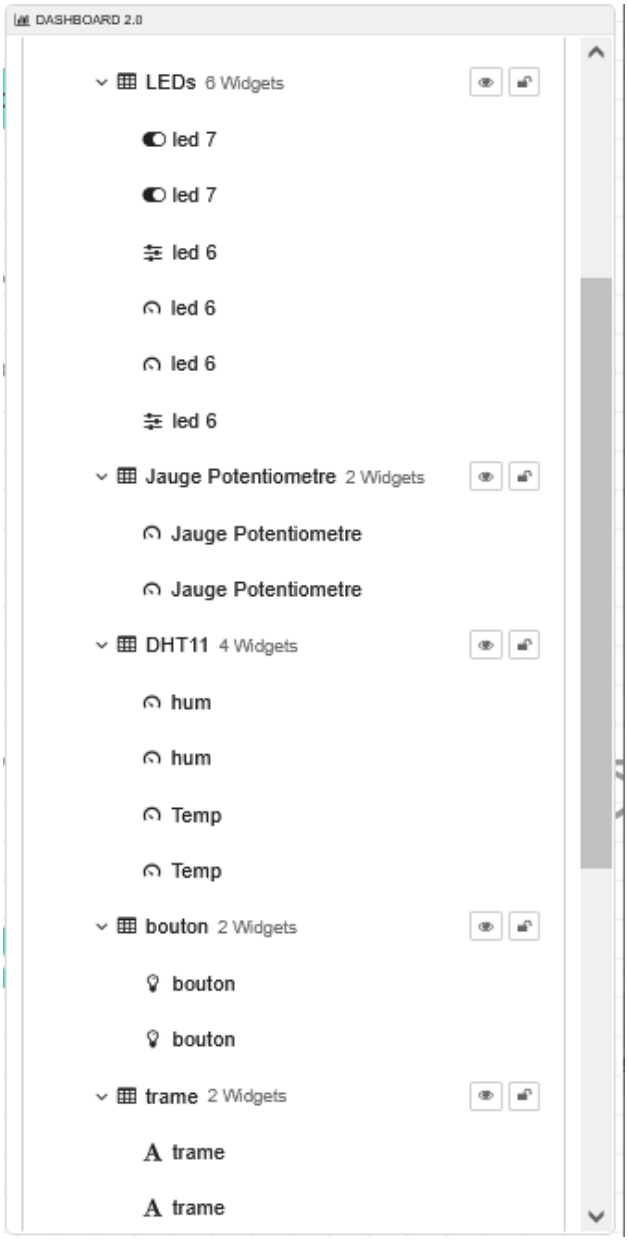
- Change → définir msg.payload



Préparation de la page web

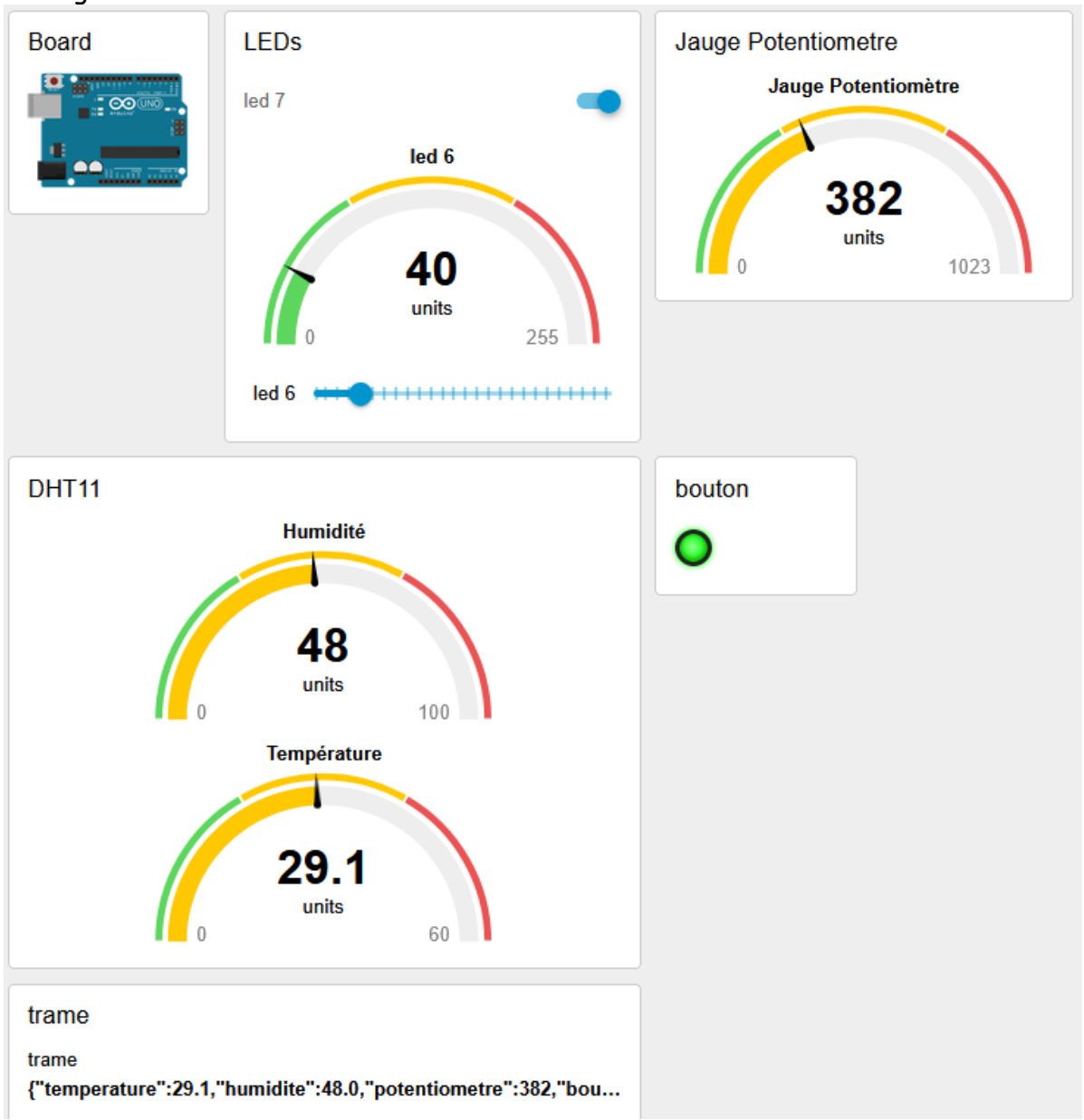
Le Layout de l'application est organisé autour d'une page nommée « Page Name » et de plusieurs groupes permettant de structurer les différents éléments du tableau de bord. Les widgets sont répartis selon leur fonction : représentation de la carte Arduino, commande des LEDs, visualisation du potentiomètre, affichage des mesures du DHT11, état du bouton et visualisation des trames échangées.


DASHBOARD 2.0
My Dashboard
Edit Settings
Open Dashboard
Layout
Theming
Client Data
Pages
Link
Page
Here you can re-order and move your widgets, groups and pages.
Page Name3 6 Groups
Board 2 Widgets
template
template
LEDs 6 Widgets
led 7
led 7
led 6
led 6
led 6
led 6
Jauge Potentiometre 2 Widgets
Jauge Potentiometre
Jauge Potentiometre
DHT11 4 Widgets
hum
hum
Temp
MESSAGES DE DÉBOGAGE
Tous les noeuds
Tout
msg.payload : Object
{ temperature: 29.6, humidite: 47,
potentiometre: 382, bouton: true }
09/08/2026 10:31:53 noeud: debug 5
led7 : msg.payload : string[32]
"{"":false,"led7":true,"led6":80}"
09/08/2026 10:31:54 noeud: debug 3
msg.payload : Object


DASHBOARD 2.0
LEDs 6 Widgets
led 7
led 7
led 6
led 6
led 6
led 6
Jauge Potentiometre 2 Widgets
Jauge Potentiometre
Jauge Potentiometre
DHT11 4 Widgets
hum
hum
Temp
Temp
bouton 2 Widgets
bouton
bouton
trame 2 Widgets
trame
trame

7.3 Organisation du dashboard

Le Layout est organisé autour d'une page unique et de plusieurs groupes, chacun dédié à une fonction : représentation de la carte Arduino, commande des LEDs, visualisation du potentiomètre, mesures du DHT11, état du bouton, trame brute échangée



Groupe	Node	Type	Fonction
Board	template	ui-template	Afficher la représentation de la carte Arduino
LEDs	led 7	ui-switch	Commander la LED 7 en tout-ou-rien
LEDs	led 7	ui-led	Visualiser l'état de la LED 7
LEDs	led 6	ui-slider	Régler la valeur PWM de la LED 6
LEDs	led 6	ui-gauge	Visualiser la valeur PWM de la LED 6

Jauge Potentiomètre	Jauge Potentiometre	ui-gauge	Afficher la valeur du potentiomètre (0-1023)
DHT11	hum / Temp	ui-gauge	Afficher l'humidité et la température mesurées
bouton	bouton	ui-led	Visualiser l'état du bouton
trame	trame	ui-text	Afficher la trame JSON reçue
Traitement	function 2	function	Construire la trame JSON de commande vers l'Arduino
Traitement	json	json	Décoder la trame JSON reçue de l'Arduino
Traitement	définir msg.payload	change	Extraire la donnée bouton pour le widget ui-led

7.4 Synthèse

7.4.1 Configuration du node serial-port

Nom : Arduino — Serial Port : le port COM de votre carte — Baud Rate : 9600, 8 bits, sans parité, 1 stop bit — Split input : sur le caractère \n.

7.4.2 Node Function 2 — construction de la trame de commande

```
let trame = flow.get('trame') || {};
trame[msg.topic] = msg.payload;
flow.set('trame', trame);

msg.payload = JSON.stringify(trame);
return msg;
```

7.4.3 Node Change — extraction pour le widget ui-led

Règle : Définir msg.payload sur la valeur msg.payload.bouton — permet d'extraire uniquement l'état du bouton depuis la trame JSON complète pour l'afficher sur le node ui-led dédié.

7.4.4 Node Template — affichage de la carte Arduino

```
<template>
  <div>
    <img src="/arduinoUno.png" style="width:100%">
  </div>
</template>
```

Pour que l'image soit servie par Node-RED, ajouter dans le fichier settings.js (répertoire C:\Users\UserName\.node-red) la ligne suivante, puis créer le répertoire correspondant et y copier l'image (fournie en Annexe C) :

```
httpStatic: 'C:\\Users\\UserName\\.node-red\\node-red-static\\',
```

Exemple de trame envoyée par l'Arduino :

```
{"temperature": 24.0,"humidite": 52.0,"potentiometre": 643,"bouton": true}
```

Exemple de trame envoyée par Node-RED : `{"led7":true,"led6":200}`

Sens	Donnée	Type
Arduino → Node-RED	température	float
Arduino → Node-RED	humidité	float
Arduino → Node-RED	potentiomètre	0 à 1023
Arduino → Node-RED	bouton D2	true / false
Node-RED → Arduino	LED 7	true / false
Node-RED → Arduino	LED 6	0 à 255

7.5 Questionnaire de synthèse

1. Pourquoi utiliser un format JSON plutôt que d'envoyer les valeurs séparément sur la liaison série ?

2. Quel est le rôle du node change dans ce flow ?

3. Pourquoi le node function2 utilise-t-il flow.get()/flow.set() plutôt qu'une simple variable JavaScript locale ?

4. Que se passerait-il si le débit (baud rate) du node serial ne correspondait pas à celui configuré côté Arduino ?

5. En quoi ce projet complète-t-il la chaîne acquisition → traitement → commande → communication déjà rencontrée en TP1 et TP2 ?

6. Quelle est la différence entre la réactivité obtenue ici (Node-RED côté PC) et celle obtenue par interruption matérielle (Timer2, partie A) ?

8 Conclusion du TP

Ce TP a complété la panoplie de mécanismes de réactivité d'un système embarqué : après la scrutation (TP1) et les interruptions externes (TP2), les interruptions internes (TP3a) vous avez construit des interface homme-machine web avec Node-RED, sans développement web classique, et en établissant une communication bidirectionnelle structurée (JSON) entre la carte Arduino et cet outil.

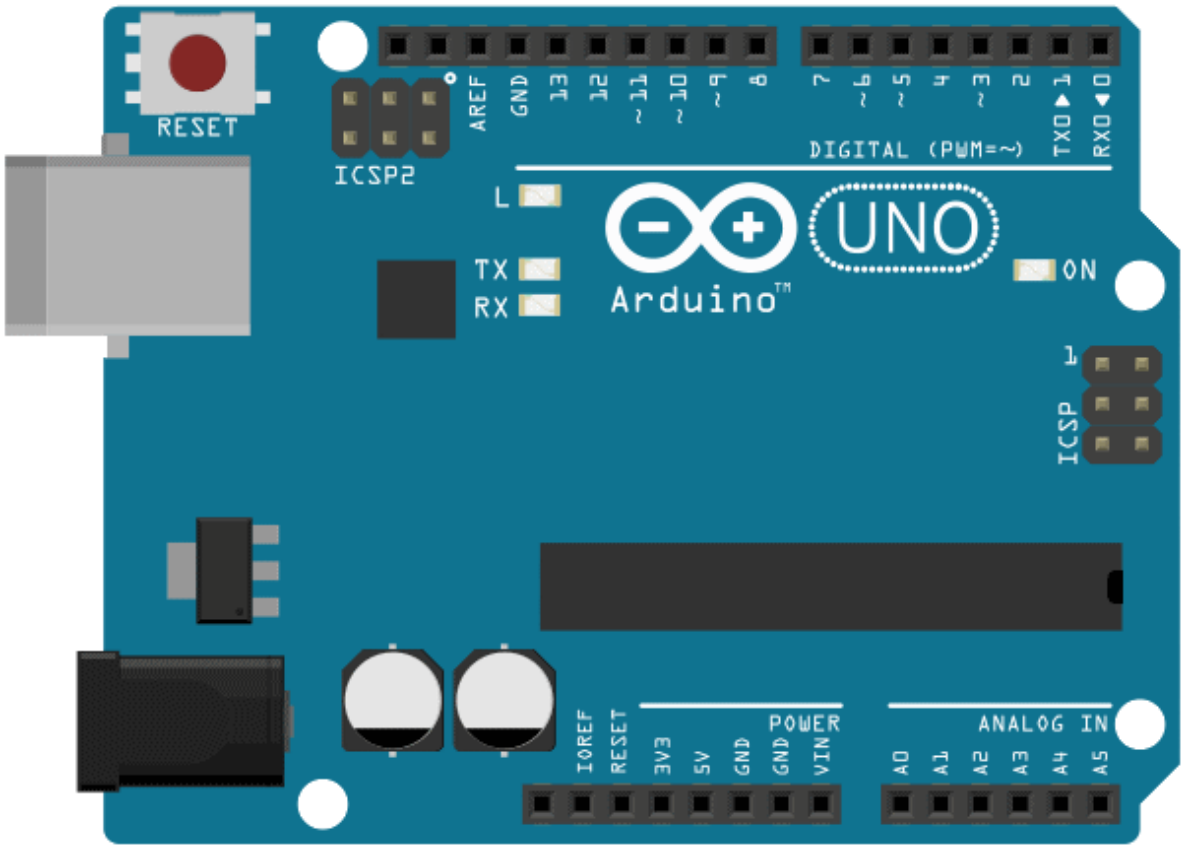
La démarche acquise —structurer des échanges de données — est réutilisable pour aborder des systèmes embarqués connectés plus complexes.

9 Annexes :

Documentation node-RED
https://nodered.org/docs/

Nœud serial port.
https://flows.nodered.org/node/node-red-node-serialport

Image du template :



10 Les éléments Node-RED

Élément	Rôle	Syntaxe	Exemple
`msg`	Message qui circule entre les nœuds	`msg.propriété`	`msg.payload`
`msg.payload`	Contenu principal du message	`msg.payload`	`msg.payload = 25;`
`msg.topic`	Identifie le sujet du message	`msg.topic`	`msg.topic = "temperature";`
`msg.payload = ...`	Modifie le contenu du message	`msg.payload = valeur;`	`msg.payload = true;`
`return msg`	Envoie le message vers la sortie de la fonction	`return msg;`	`return msg;`

Élément	Rôle	Syntaxe	Exemple
`flow.get()`	Récupère une donnée mémorisée dans le contexte `flow`	`flow.get("nom")`	`flow.get("trame")`
`flow.set()`	Mémorise une donnée dans le contexte `flow`	`flow.set("nom", valeur)`	`flow.set("trame", trame)`

Dans **Node-RED**, msg est le message qui circule entre les nœuds.

Élément	Définition	Exemple
`msg.topic`	Indique **le sujet, le nom ou l'identifiant** de la donnée transportée.	`"temperature"`
`msg.payload`	Contient **la donnée ou la valeur transportée** par le message.	`24.6`

Exemple :

```
msg.topic = "temperature";
msg.payload = 24.6;

return msg;
```

Code	Rôle	Exemple
`msg`	Message Node-RED	`msg.payload`
`msg.payload`	Donnée principale du message	`msg.payload = 25`
`msg.topic`	Nom/sujet de la donnée	`"temperature"`
`return msg`	Envoie le message suivant	`return msg;`
`let`	Déclare une variable	`let vitesse = 850;`
`{}`	Objet vide	`let trame = {};`
`[]`	Tableau vide	`let valeurs = [];`
`=`	Affectation	`x = 10`
`===`	Comparaison stricte	`x === "true"`
`if`	Condition	`if (x > 10)`
`else if`	Deuxième condition	`else if (x === 0)`
`.toString()`	Convertit en texte	`x.toString()`
`.trim()`	Supprime espaces début/fin	`texte.trim()`
`JSON.stringify()`	Objet → texte JSON	`JSON.stringify(trame)`
`JSON.parse()`	Texte JSON → objet	`JSON.parse(texte)`
`objet["clé"]`	Accès à une donnée	`trame["temperature"]`
`objet["clé"] = valeur`	Ajoute/modifie une donnée	`trame["temperature"] = 24.6`

Code	Rôle	Exemple
`flow.get()`	Récupère une donnée mémorisée	`flow.get("trame")`
`flow.set()`	Mémorise une donnée	`flow.set("trame", trame)`
`		`
`+`	Concaténation	`"Temp : " + temperature`